PySpark Skill Booster Complete Guide

Sachin Chandrashekhar

Founder, Data Engineering Hub™

https://sachinchandrashekhar.com

Follow us on social media for exciting updates and exclusive content!

YouTube: https://www.youtube.com/@MentorSachin

Instagram: https://www.instagram.com/mentorsachin/

Website: https://dataengineeringhub.in

legal@dataengineeringhub.in www.dataengineeringhub.in

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Copyright & Usage Notice: This document and its supporting artifacts are copyrighted and intended strictly for personal use only.

- This document is reserved exclusively for individuals enrolled in Data Engineering courses by Sachin Chandrashekhar.

- Please do not share this document or upload it publicly. It is intended for personal use only.

- If you have obtained this document for free from any website other than the official course website, or from social media or public git repositories, please contact **legal@dataengineeringhub.i**

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

📚 Course Overview

🎯 This PySpark Skill Booster course provides practical foundation for AWS data engineering projects. The course focuses on hands-on learning with AWS Glue notebooks, preparing you for real-world ETL development.

🎓 Prerequisites

- ✅ Python fundamentals and Lambda functions
- ✅ SQL query skills
- ✅ AWS account with access to Glue and S3

🎯 Learning Objectives

- • Read and write data in various formats (CSV, Parquet)
- • Transform data using PySpark operations
- • Perform aggregations and joins on large datasets
- • Handle NULL values and data quality issues
- • Prepare for PySpark interview questions

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

⚙️ Setup & Environment

📝 Step 1: Install Visual Studio Code

Download and install VS Code from https://code.visualstudio.com/

☁️ Step 2: AWS CloudShell Setup

1️⃣ Open AWS Console and search for "CloudShell"

2️⃣ Click "Open US East 1 Environment"

3️⃣ Upload the setup scripts provided in the course

🚀 Step 3: Run Setup Script

Execute these commands in CloudShell:

```
chmod +x *
sh mission-deh-hof-setup.sh
```

✅ This creates:

- • S3 bucket: mission-deh-hof-{account_id}
- • IAM role: mission-deh-hof-glue-role
- • Sample datasets in S3 (customers, orders, products)

📓 Step 4: Create AWS Glue Notebook

1️⃣ Navigate to AWS Glue → ETL Jobs → Notebooks

2️⃣ Click "Create notebook"

3️⃣ Select IAM role: mission-deh-hof-glue-role

4️⃣ Configure settings:

- • Idle timeout: 30 minutes
- • Number of workers: 2

5️⃣ Wait for notebook to start

💡 Important: Iterative Development

🔄 Glue notebooks allow you to run code cell-by-cell, test output immediately, and iteratively develop your PySpark scripts. Always stop notebooks when not in use to avoid charges!

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

📊 Module 1: DataFrame Creation & Schema Definition

🎯 Key Concepts

• DataFrames are PySpark's primary data structure (like SQL tables)

• Schema defines column names and data types

• Schema inference vs explicit schema definition

• Always use explicit schemas in production (interview topic!)

📖 Reading CSV with Schema Inference

```
import boto3
from pyspark.sql.functions import col

account_id = boto3.client('sts').get_caller_identity()['Account']
bucket_name = f"mission-deh-hof-{account_id}"

# Read CSV with automatic schema inference
customers_df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv(f"s3://{bucket_name}/raw/customers.csv")

customers_df.show(5)
customers_df.printSchema()
```

✅ Defining Explicit Schema (DDL String)

⭐ Interview Topic: Why explicit schemas in production?

Answer: Schema inference requires Spark to scan the entire file to determine data types. For large files (GBs/TBs), this is inefficient. Explicit schemas skip this step and improve performance.

```
# Define schema using DDL string (recommended for production)
customers_schema = "customer_id INT, name STRING, email STRING, country STRING,
signup_date STRING"

customers_df = spark.read \
    .option("header", "true") \
    .schema(customers_schema) \
    .csv(f"s3://{bucket_name}/raw/customers.csv")

customers_df.printSchema()
```

📋 Common Data Types

Data Type	Description

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

INT	Integer numbers
LONG	Large integer numbers
DOUBLE	Decimal numbers
STRING	Text data
DATE	Date values
TIMESTAMP	Date and time values
BOOLEAN	True/False values

🎯 Basic DataFrame Operations

```
# Count rows
print(f"Total rows: {customers_df.count()}")

# Show column names
print(f"Columns: {customers_df.columns}")

# Show first few rows
customers_df.show(5)
```

💪 Exercise

📝 Read orders.csv with explicit schema:

```
# Schema: order_id INT, customer_id INT, product_id INT,
#         quantity INT, order_date STRING, amount DOUBLE

# Your code here:
orders_schema = "order_id INT, customer_id INT, product_id INT, quantity INT,
order_date STRING, amount DOUBLE"
orders_df = spark.read.option("header",
"true").schema(orders_schema).csv(f"s3://{bucket_name}/raw/orders.csv")
orders_df.printSchema()
orders_df.show(5)
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

💾 Module 2: Reading & Writing Data (Parquet Format)

🎯 Why Parquet?

⭐ Interview Topic: Why is Parquet preferred for data lakes?

Feature	Benefit
Columnar Storage	Faster queries when selecting specific columns
Compression	Smaller file sizes (saves storage costs)
Schema Embedded	No need to specify schema when reading
Optimized for Analytics	Better performance for aggregations and filters

📝 Writing Data as Parquet

```
# Write DataFrame to S3 as Parquet
customers_df.write \
    .mode("overwrite") \
    .parquet(f"s3://{bucket_name}/curated/customers_parquet/")

print("✅ Data written to curated folder")
```

📖 Reading Parquet Files

✨ Notice: No need to specify header or schema options!

```
# Read Parquet file (schema is embedded)
customers_parquet_df =
spark.read.parquet(f"s3://{bucket_name}/curated/customers_parquet/")

customers_parquet_df.show(5)
customers_parquet_df.printSchema()
```

🔄 Write Modes

⭐ Interview Topic: What are the different write modes in PySpark?

Write Mode	Behavior
overwrite	Replace existing data completely
append	Add new data to existing data
ignore	Skip writing if data already exists
error (default)	Throw error if data exists

📁 Data Lake Structure

📂 Raw vs Curated folders:

- • raw/ - Original source data (unchanged)
- • curated/ - Processed/transformed data (clean, ready for analytics)

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

💪 Exercise

📝 Write products_df as Parquet and read it back:

```
# Write products as Parquet
products_df.write.mode("overwrite").parquet(f"s3://{bucket_name}/curated/produc
ts_parquet/")

# Read back
products_parquet_df =
spark.read.parquet(f"s3://{bucket_name}/curated/products_parquet/")
products_parquet_df.printSchema()
products_parquet_df.show(5)
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🔄 Module 3: Transformations

🎯 Core Transformation Operations

PySpark transformations are similar to SQL operations. The syntax differs, but the underlying concepts remain the same.

1️⃣ Select - Choosing Specific Columns

```
# Method 1: Direct column names
customers_df.select("customer_id", "name", "country").show(5)

# Method 2: Using col() function
from pyspark.sql.functions import col
customers_df.select(col("customer_id"), col("email")).show(5)
```

2️⃣ Filter - Filtering Rows

```
# Filter by single condition
customers_df.filter(col("country") == "USA").show()

# Filter by numeric condition
orders_df.filter(col("amount") > 150).show()

# Multiple conditions with AND
orders_df.filter((col("amount") > 100) & (col("quantity") >= 2)).show()

# Multiple conditions with OR
customers_df.filter((col("country") == "USA") | (col("country") ==
"India")).show()
```

⚠️ Important: Each condition must be in separate parentheses when using & or |

3️⃣ WithColumn - Adding/Modifying Columns

⭐ Interview Topic: How to add columns with constant values?

```
from pyspark.sql.functions import lit

# Add column with constant value
customers_df.withColumn("status", lit("Active")).show(5)

# Add calculated column
products_df.withColumn("price_with_tax", col("price") * 1.1).show()

# Transform existing column to uppercase
from pyspark.sql.functions import upper
customers_df.withColumn("name_upper", upper(col("name"))).show(5)
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

4️⃣ Conditional Columns with when/otherwise

⭐ Interview Topic: Creating conditional columns

```
from pyspark.sql.functions import when

# Categorize orders by amount
orders_df.withColumn(
    "order_category",
    when(col("amount") > 200, "High")
    .when(col("amount") > 100, "Medium")
    .otherwise("Low")
).select("order_id", "amount", "order_category").show()
```

5️⃣ Drop - Removing Columns

```
# Drop single column
customers_df.drop("email").show(5)

# Drop multiple columns
customers_df.drop("email", "signup_date").show(5)
```

6️⃣ WithColumnRenamed - Renaming Columns

```
# Rename column
customers_df.withColumnRenamed("name", "customer_name").show(5)
```

🔗 Chaining Transformations

⭐ Interview Topic: How to chain multiple operations?

```
# Chain multiple transformations
result = orders_df \
    .filter(col("amount") > 100) \
    .withColumn("discount", col("amount") * 0.1) \
    .withColumn("final_amount", col("amount") - col("discount")) \
    .select("order_id", "amount", "discount", "final_amount")

result.show()
```

💪 Exercise

📝 Using products_df:

1. Filter products in "Electronics" category

2. Add "discounted_price" column (20% off)

3. Add "stock_status" column ("In Stock" if stock_quantity > 100, else "Low Stock")

4. Select only product_name, price, discounted_price, stock_status

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

legal@dataengineeringhub.in www.dataengineeringhub.in

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

📊 Module 4: Aggregations

🎯 Why Aggregations?

Aggregations help extract meaningful insights from raw data. For example: total revenue by product, average order value, customer spending patterns.

1️⃣ Simple Count

```
# Total number of records
total_orders = orders_df.count()
print(f"Total orders: {total_orders}")
```

2️⃣ GroupBy with Count

```
# Count customers by country
customers_df.groupBy("country").count().show()

# Count orders by product
orders_df.groupBy("product_id").count().show()
```

3️⃣ Sum Aggregation

```
from pyspark.sql.functions import sum

# Total revenue by product
orders_df.groupBy("product_id") \
    .agg(sum("amount").alias("total_revenue")) \
    .show()

# Total quantity sold by product
orders_df.groupBy("product_id") \
    .agg(sum("quantity").alias("total_quantity")) \
    .show()
```

4️⃣ Average, Min, Max

```
from pyspark.sql.functions import avg, min, max, round

# Average order amount by customer (rounded to 2 decimals)
orders_df.groupBy("customer_id") \
    .agg(round(avg("amount"), 2).alias("avg_amount")) \
    .show()

# Min and Max price by category
products_df.groupBy("category") \
    .agg(
        min("price").alias("min_price"),
        max("price").alias("max_price")
    ).show()
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

5️⃣ Multiple Aggregations

```
from pyspark.sql.functions import count

# Multiple aggregations on orders by customer
customer_summary = orders_df.groupBy("customer_id").agg(
    count("order_id").alias("total_orders"),
    sum("amount").alias("total_spent"),
    round(avg("amount"), 2).alias("avg_order_value"),
    max("amount").alias("max_order_value")
)

customer_summary.show()
```

6️⃣ Sorting Results

```
# Sort by total spent (descending)
customer_summary.orderBy(col("total_spent").desc()).show()

# Sort by revenue (ascending)
revenue_by_product.orderBy("total_revenue").show()
```

7️⃣ Count Distinct

```
from pyspark.sql.functions import countDistinct

# Count unique customers who placed orders
orders_df.agg(countDistinct("customer_id").alias("unique_customers")).show()
```

8️⃣ Aggregation Without GroupBy

```
# Overall statistics for all orders
overall_stats = orders_df.agg(
    count("order_id").alias("total_orders"),
    sum("amount").alias("total_revenue"),
    round(avg("amount"), 2).alias("avg_order_value"),
    min("amount").alias("min_order"),
    max("amount").alias("max_order")
)

overall_stats.show()
```

💪 Exercise

📝 Using orders_df, group by product_id and calculate:

- Total number of orders
- Total quantity sold
- Total revenue

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- Average order amount

Then sort by total revenue (descending)

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🔗 Module 5: Joins

🎯 Join Types Overview

⭐ Interview Topic: Explain different join types and when to use them

Join Type	Returns
inner	Only matching rows from both DataFrames
left	All rows from left DataFrame + matching from right
right	All rows from right DataFrame + matching from left
full/outer	All rows from both DataFrames

1️⃣ Inner Join

```
# Join orders with customers (inner join)
orders_with_customers = orders_df.join(
    customers_df,
    orders_df.customer_id == customers_df.customer_id,
    "inner"
)

orders_with_customers.select(
    orders_df.order_id,
    customers_df.name,
    customers_df.country,
    orders_df.amount
).show()

# Note: Orders with NULL customer_id are excluded
print(f"Total orders: {orders_df.count()}")
print(f"After inner join: {orders_with_customers.count()}")
```

2️⃣ Left Join

```
# Keep all orders, even without customer match
orders_left_join = orders_df.join(
    customers_df,
    orders_df.customer_id == customers_df.customer_id,
    "left"
)

# All orders are retained (including those with NULL customer_id)
print(f"After left join: {orders_left_join.count()}")
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

3️⃣ Right Join

```
# Keep all customers, even without orders
customers_right_join = orders_df.join(
    customers_df,
    orders_df.customer_id == customers_df.customer_id,
    "right"
)

# All customers are retained
print(f"After right join: {customers_right_join.count()}")
```

4️⃣ Joining Multiple DataFrames

```
# Chain multiple joins
complete_orders = orders_df \
    .join(customers_df, orders_df.customer_id == customers_df.customer_id,
"inner") \
    .join(products_df, orders_df.product_id == products_df.product_id, "inner")

complete_orders.select(
    orders_df.order_id,
    customers_df.name.alias("customer_name"),
    products_df.product_name,
    orders_df.quantity,
    orders_df.amount
).show()
```

5️⃣ Finding Unmatched Records

⭐ Very Common Interview Question: Find customers without orders

```
# Use left join + filter for NULL
customers_without_orders = customers_df.join(
    orders_df,
    customers_df.customer_id == orders_df.customer_id,
    "left"
).filter(col("order_id").isNull())

customers_without_orders.select(
    customers_df.customer_id,
    customers_df.name,
    customers_df.country
).show()
```

💪 Exercise

📝 Join orders with products (inner join), calculate total revenue by category, and sort by revenue (descending)

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

❓ Module 6: Handling NULL Values

🎯 Understanding NULL

⭐ Interview Topic: NULL literal "NULL" vs actual NULL value

In CSV files, the string "NULL" is different from an actual NULL value. Use option("nullValue", "NULL") to tell Spark that "NULL" strings should be treated as actual NULLs.

1️⃣ Reading Data with NULL Handling

```
# Specify that "NULL" strings should be treated as actual NULL values
orders_df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("nullValue", "NULL") \
    .csv(f"s3://{bucket_name}/raw/orders.csv")

orders_df.show()
```

2️⃣ Identifying NULL Values

```
# Filter rows where customer_id is NULL
orders_df.filter(col("customer_id").isNull()).show()

# Check products with NULL stock_quantity
products_df.filter(col("stock_quantity").isNull()).show()
```

3️⃣ Excluding Rows with NULL Values

```
# Method 1: Using isNotNull()
orders_no_null = orders_df.filter(col("customer_id").isNotNull())
print(f"Original: {orders_df.count()}, After excluding:
{orders_no_null.count()}")

# Method 2: Using dropna()
orders_dropped = orders_df.dropna(subset=["customer_id"])
print(f"After dropna: {orders_dropped.count()}")
```

4️⃣ Replacing NULL with Default Values

```
# Replace NULL stock_quantity with 0
products_filled = products_df.fillna({"stock_quantity": 0})
products_filled.show()

# Replace NULL customer_id with 0
orders_filled = orders_df.fillna({"customer_id": 0})
orders_filled.show()
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🛠️ Troubleshooting NULL Issues

If fillna() doesn't work as expected:

- 1. Check if you used option("nullValue", "NULL") when reading
- 2. Verify the column name is correct
- 3. Ensure the replacement value matches the column data type

💪 Exercise

📝 Using products_df:

1. Identify rows where stock_quantity is NULL

2. Create DataFrame excluding those rows

3. Create DataFrame replacing NULL with 0

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🔍 Module 7: Distinct, Sorting & String Functions

1️⃣ Finding Distinct Values

```
# Get distinct countries
customers_df.select("country").distinct().show()

# Get distinct product categories
products_df.select("category").distinct().show()
```

2️⃣ Removing Duplicate Rows

⭐ Interview Topic: dropDuplicates() based on all columns vs specific columns

```
# Remove duplicates based on ALL columns
unique_customers = customers_df.dropDuplicates()
print(f"Original: {customers_df.count()}, After dedup:
{unique_customers.count()}")

# Remove duplicates based on specific column (keep first occurrence)
unique_by_country = customers_df.dropDuplicates(["country"])
unique_by_country.select("customer_id", "name", "country").show()
```

3️⃣ Sorting Data

```
# Sort ascending (default)
orders_df.orderBy("amount").select("order_id", "amount").show(5)

# Sort descending
orders_df.orderBy(col("amount").desc()).select("order_id", "amount").show(5)

# Sort by multiple columns
orders_df.orderBy(col("product_id"), col("amount").desc()).show(10)
```

4️⃣ String Functions - Upper and Lower

```
from pyspark.sql.functions import upper, lower

# Convert to uppercase
customers_df.withColumn("name_upper", upper(col("name"))).select("name",
"name_upper").show(5)

# Convert to lowercase
customers_df.withColumn("country_lower",
lower(col("country"))).select("country", "country_lower").show(5)
```

5️⃣ String Functions - Concat

```
from pyspark.sql.functions import concat, lit

# Concatenate columns with literal separator
```


© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

```
customers_df.withColumn(
    "full_info",
    concat(col("name"), lit(" - "), col("country"))
).select("name", "country", "full_info").show(5)
```

6️⃣ String Functions - Split

⭐ Common Interview Question: Extract domain from email

```
from pyspark.sql.functions import split

# Split email by "@"
customers_df.withColumn(
    "email_parts",
    split(col("email"), "@")
).select("email", "email_parts").show(5, truncate=False)

# Extract domain (second part after split)
customers_df.withColumn(
    "domain",
    split(col("email"), "@")[1]
).select("email", "domain").show(5)
```

💡 Note: After split(), result is a list. Use [0] for first element, [1] for second, etc.

💪 Exercise

📝 Using products_df:

1. Get distinct categories

2. Sort by price (descending)

3. Create uppercase product_name column

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🧠 Module 8: PySpark Fundamentals - Theory & Background

🎯 What is PySpark?

PySpark is the Python API for Apache Spark - a distributed computing framework for processing large datasets.

💡 Why Use PySpark?

- ✅ Process large datasets that don't fit in memory
- ✅ Distributed computing - automatically splits work across machines
- ✅ Fast - In-memory processing (100x faster than MapReduce)
- ✅ Easy to use - Python API similar to Pandas
- ✅ AWS Glue uses PySpark - Standard for ETL jobs on AWS

🏗️ Spark Architecture

⭐ Interview Topic: How does Spark process data across multiple machines?

Spark uses a cluster of machines (workers/nodes) to process data in parallel. YARN (Yet Another Resource Negotiator) manages resources across these machines. As a developer, you write code as if working with one machine - Spark handles the distribution automatically.

🔑 Key Concepts for AWS Glue

📌 SparkSession

- • Entry point to Spark functionality
- • Available as "spark" variable in Glue notebooks
- • Use it to read/write data: spark.read.csv()

📌 GlueContext

- • AWS Glue-specific wrapper around SparkContext
- • Provides additional Glue features (DynamicFrames, job bookmarks, etc.)
- • Available as "glueContext" variable in Glue notebooks

📌 DPU (Data Processing Unit)

- • Glue's unit of processing power
- • 1 DPU = 4 vCPU + 16 GB memory

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- • You pay per DPU-hour (serverless pricing)

🔄 Common PySpark Workflow

```
# 1. Read data from S3
df = spark.read.parquet("s3://bucket/data/")

# 2. Transform data
df_filtered = df.filter(col("status") == "active")
df_transformed = df_filtered.withColumn("new_col", col("old_col") * 2)

# 3. Aggregate
df_agg = df_transformed.groupBy("category").agg(sum("amount"))

# 4. Write results
df_agg.write.mode("overwrite").parquet("s3://bucket/aggregated/")
```

💰 Cost Optimization Tips

- ⚠️ Always stop Glue notebooks when not in use!
- ✅ Set idle timeout to 30 minutes
- ✅ Use minimum 2 workers for learning
- ✅ Monitor job runs and optimize DPU allocation

📚 Going Deeper

This course provides foundation for AWS data engineering projects. For advanced Spark optimization, performance tuning, and production best practices, consider the comprehensive Spark deep dive course (1+ month dedicated study) which is part of Diamond Membership.

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🧹 Cleanup Your Resources

⚠️ Important: Avoid Unnecessary Charges

After completing your PySpark practice, clean up all AWS resources to avoid charges:

1️⃣ Run Cleanup Script

In CloudShell:

```
ls
sh mission-deh-hof-cleanup.sh
```

✅ This will delete:

- • S3 bucket and all contents
- • IAM role and policies

2️⃣ Terminate Glue Notebooks

Go to AWS Glue Console → ETL Jobs → Notebooks

✅ Verify no yellow warning appears (indicates running notebooks)

✅ If warning exists, stop all running notebooks

3️⃣ Delete CloudShell Environment

In CloudShell:

- • Click Actions → Delete
- • Confirm deletion

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

🎓 What's Next?

✅ You've Completed the PySpark Skill Booster!

You now have practical foundation in:

- ✅ Reading and writing data in various formats
- ✅ Transforming data with PySpark operations
- ✅ Performing aggregations and joins
- ✅ Handling NULL values and data quality
- ✅ Working with AWS Glue notebooks

🚀 Next Steps

1️⃣ Move to the Labs Course

- • Build a production-grade AWS data engineering project
- • Apply everything you've learned in real-world scenario
- • Work with actual datasets and requirements

2️⃣ Take the Glue Deep Dive Course

- • Learn advanced Glue features
- • Understand Glue Python Shell
- • Master production deployment strategies

3️⃣ Consider Diamond Membership for Advanced Spark

- • 1+ month dedicated to Spark mastery
- • Performance optimization and tuning
- • Advanced interview preparation

💪 Keep Practicing!

🎯 Consistency is key! Practice 15-20 minutes daily on your weak areas. The journey from data engineer to architect takes time, but with persistence, you will succeed!

📚 Remember:

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- • Skills pay the bills - keep adding new skills
- • Don't give up when things get tough
- • Follow the learning path sequentially
- • Target the Data Engineering Hub Hall of Fame!

Follow us on social media for exciting updates and exclusive content!

YouTube: https://www.youtube.com/@MentorSachin

Instagram: https://www.instagram.com/mentorsachin/

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution